Singly Linked List Data Structure

(https://github.com/learn-co-curriculum/python-p3-dsa-singly-linked-list) (https://github.com/learn-co-curriculum/python-p3-dsa-singly-linked-list/issues/new)

Learning Goals

- Identify the use cases for a Singly Linked List.
- Demonstrate common methods for a Singly Linked List.
- Differentiate between a Singly Linked List and a list.

Key Vocab

- **Sequence**: a data structure in which data is stored and accessed in a specific order.
- **Stack** is a linear data structure that follows the principle of Last In First Out (LIFO).
- **Index**: the location, represented by an integer, of an element in a sequence.
- **Iterable**: able to be broken down into smaller parts of equal size that can be processed in turn. You can loop through any iterable object.
- **Slice**: a group of neighboring elements in a sequence.
- **List**: a mutable data type in Python that can store many types of data. The most common data structure in Python.
- **Tuple**: an immutable data type in Python that can store many types of data.
- **Range**: a data type in Python that stores integers in a fixed pattern.
- **String**: an immutable data type in Python that stores unicode characters in a fixed pattern. Iterable and indexed, just like other sequences.

Introduction

In this lesson, we'll learn what a **Singly Linked List** is, along with how to build a `LinkedList` class. We'll also learn some of its common methods to get an understanding of what the differences between a Singly Linked List and an list are.

What Is a Linked List?

A linked list is a linear (ordered) collection of data that consists of several elements, called **nodes**, with each node pointing to the next node in the list. Unlike lists, linked lists aren't indexed.

Think of it like a train: each car is connected to the next car, which is connected to the next car, and so forth. In an list, we can say "Give me the sixth element," but in a linked list, we have to start at the beginning of the train, and go from the first car, to the second car, and so forth:

The nodes in a linked list each have a value, and a pointer to another node, otherwise pointing to nil if it is at the end of the list.

Why Linked Lists?

Linked lists and lists are both data structures that can hold ordered lists of data. Depending on what kind of operations you need to perform with that list, there are some scenarios where a linked list can be more efficient than an list, such as efficiently adding and removing elements from any arbitrary position within the list.

Think about if we have a sorted dog list of 5 elements representing dog breeds:

```
["Bulldog", "Chihuahua", "German Shepard", "Retriever", "Shiba Inu"]
   [0]          [1]            [2]             [3]          [4]
```

After creating our list, we realize we forgot to add one dog in our list, a Chow Chow! To fix our list, we would need to *insert* the "Chow Chow" element into the list in the correct index, which would be 2. Because there is already an element in the 2nd index, and more elements in the sequential indexes, all of those elements would have to be shifted down a spot, and given a new index.

Since this is a smaller list, it doesn't seem like the biggest deal to move the last 3 elements down a place, but as you can imagine, if we had an list of hundreds or thousands or even millions of elements, re-indexing *all* of those elements would take a really long time! This is where linked lists come in handy.

Defining a Singly Linked List

Time to build our custom data structure! Since our `LinkedList` class is going to contain a series of nodes linked together, we'll start by creating `Node` class:

```
class Node:
  def __init__(self, data, next_node = None):
    self.data = data
    self.next_node = next_node
```

Each node needs to keep track of some data, as well as a reference to the next node in the list.

Next, we can build out a `LinkedList` class, with an `initialize` method where we declare an instance variable for the `head` of the linked list:

```
class LinkedList:
  def __init__(self, head = None):
    self.head = head
```

The `head` node is going to be the very first node in our linked list, and will point to the next node once we start adding more elements.

Adding Nodes

Let's say we want to recreate the data structure of dogs we had before (`["Bulldog", "Chihuahua", "German Shepard", "Retriever", "Shiba Inu"]`). This time we'll use a linked list instead of an list. The simplest way to do this is to create a series of nodes and link them together using the `next_node` attribute:

```
bulldog = Node("Bulldog")
# Bulldog
```

```
chihuahua = Node("Chihuahua")
bulldog.next_node = chihuahua
# Bulldog -> Chihuahua
german_shepard = Node("German Shepard")
chihuahua.next_node = german_shepard
# Bulldog -> Chihuahua -> German Shepard
```

While this technically qualifies as a linked list, it's not the most pleasant to work with! We can make our data structure a bit more developer-friendly by using the `LinkedList` class to build a list by creating an `append` method in our `LinkedList` class.

The `append` method should add a node to the `head` of the list if the list is empty, and add it to the end of the list if not:

```
class LinkedList:

  def __init__(self, head = None):
    self.head = head

  def append(self, node):
    # Add element to the beginning of the list if the list is empty
    if self.head == None:
        self.head = node
        return
    # Otherwise, traverse the list to find the last node
    last_node = self.head
    while last_node.next_node:
      last_node = last_node.next_node
    # and add the node to the end
    last_node.next_node = node
```

Now we can build our linked list like so:

```
list = LinkedList()
```

```
list.append(Node("Bulldog"))
# Bulldog
list.append(Node("Chihuahua"))
# Bulldog -> Chihuahua
list.append(Node("German Shepard"))
# Bulldog -> Chihuahua -> German Shepard
```

When to use a Singly Linked List

Linked Lists are ideal for situations when you need quick insertion and deletion, but are more expensive than lists when it comes to searching, since lists are indexed. The Big O for both insertion as well as deletion at a known node in a linked list is `O(1)` because we don't need to update indexes for the other elements in the list when a new element is added: we just need to adjust which node the `next_node` points to. With an list, insertion and deletion from anywhere other than the end are `O(n)` , because other elements need to be reindexed.

Conclusion

We use linked lists because they can be less expensive than lists when it comes to insertion and deletion within lists. Linked Lists are a very common interview data structure, so make sure you get to know them! In the next lesson, we'll build more methods in our `LinkedList` class.

Resources

- **Linked list (https://en.wikipedia.org/wiki/Linked_list)**